Министерство науки и высшего образования
Российской Федерации

Ярославский государственный университет
им. П. Г. Демидова

Кафедра математического моделирования

М. А. Погребняк

Многопоточные вычисления на основе технологий MPI и OpenMP

Учебное пособие

Черновик
Погребняк М. А.

Ярославль
Филигрань
2025

УДК 004.272.2(075.8)
ББК 32.973.14
П 43

Рецензент
кафедра математического моделирования
Ярославского государственного университета
им. П. Г. Демидова

Погребняк Максим Анатольевич.
П 43 Многопоточные вычисления на основе технологий MPI и OpenMP : учебное пособие / М. А. Погребняк ; Яросл. гос. ун-т им. П. Г. Демидова. — Ярославль : Филигрань, 2025. – 80 с.

Учебное пособие посвящено современным методам разработки параллельных программ на языке C++ с использованием технологий MPI и OpenMP — ключевых инструментов построения высокопроизводительных вычислительных систем. Распараллеливание рассматривается на примере численных алгоритмов: описаны подходы к их эффективной реализации и приведены различные варианты программной реализации.

Предназначено для студентов, обучающихся по дисциплине «Многопоточные вычисления на основе технологий MPI и OpenMP».

УДК 004.272.2(075.8)
ББК 2.973.14

© Погребняк М.А., 2025

Черновик Погребняк М.А.

Оглавление

Введение **5**

Основы параллельных вычислений **9**
Принципы декомпозиции вычислительных задач . . . 9
Методы разработки параллельных программ 12
Оценка эффективности параллельных алгоритмов . . 13

Архитектуры и принципы построения параллельных вычислительных систем **17**

Параллельное программирование с использованием MPI **21**
Общие сведения о технологии MPI 21
Установка и настройка MPI на Windows с интеграцией в Visual Studio 25
Базовые функции MPI 26
Работа с коммуникаторами в MPI 29
Взаимодействие типа точка-точка в MPI 33
Коллективные операции в MPI 40

Параллельное программирование с использованием OpenMP **45**
Общие сведения о технологии OpenMP 45
Настройка OpenMP на Windows с интеграцией в Visual Studio 48
Библиотечные функции OpenMP 49
Директивы и модель данных в OpenMP 50
Директива `parallel` 52

Черновик Потребняк М. А.

Директивы `critical`, `atomic`, `single`, `master` 59
Директива `for` 62

Методы параллельной реализации численных алгоритмов 67
Перемножение матриц 67
Численное интегрирование 68
Уравнение теплопроводности 70

Вопросы для самопроверки 73

Список литературы 76

Черновик
Погребняк М. А.

Введение

С момента появления компьютеров прошло чуть более полувека, за который область их применения охватила практически все сферы человеческой деятельности — от науки и промышленного проектирования до повседневного быта и развлечений [1].

Как и компьютеры, параллельные вычисления изначально применялись в наукоемких отраслях — атомной энергетике, аэрокосмической промышленности, численном моделировании сложных физических процессов. Сегодня они востребованы и в прикладных сферах: анализе данных, компьютерной графике, видеоиграх, социальных сетях, системах искусственного интеллекта и бизнес-приложениях. Рост объемов информации и усложнение алгоритмов делают технологии параллельных вычислений необходимыми повсюду, где важны скорость и масштабируемость [2].

Особенно востребованы технологии параллельных вычислений в современной вычислительной математике, физике, инженерии, моделировании и разработке высоконагруженных систем. Параллельные вычисления позволяют не только обрабатывать большие объемы информации, но и эффективно использовать ресурсы современных многопроцессорных систем, обеспечивая ускорение обработки данных и сокращение времени выполнения сложных задач [2].

Термин **параллельный** означает одновременное и независимое выполнение частей задачи. Это понятие связано с распределением вычислений между несколькими вычислительными устройствами. В настоящее время различают несколько уровней параллелизма [3].

Черновик
Погребняк М. А.

Параллелизм на уровне задач. Предполагает разбиение задачи на подзадачи, выполняемые одновременно на разных процессорах или потоках.

Параллелизм на уровне данных. Используется для однородных операций, когда одна и та же операция выполняется над большим объемом данных.

Параллелизм на уровне алгоритмов. Заключается в преобразовании последовательных алгоритмов в параллельные.

Параллелизм на уровне инструкций. Реализуется на аппаратном и программном уровнях через одновременное выполнение команд процессором.

Параллелизм на уровне битов. Сводится к преобразованию всех вычислений к битовым операциям.

Механизмы параллельных вычислений присутствуют практически во всех современных языках программирования, но реализованы они по-разному. В C++ используются такие технологии, как MPI [4], OpenMP [5], Intel TBB [6] и стандартные потоки из библиотеки `thread` [7]. В C# доступны средства параллельного программирования через библиотеку Task Parallel Library и `async/await` [8]. В Java реализована модель многопоточности через пакеты `java.util.concurrent` [9]. Даже в скриптовых и функциональных языках, таких как Python или Erlang, предусмотрены подходы к распараллеливанию, хотя они имеют свои особенности и ограничения [10].

Параллельные вычисления направлены на повышение производительности программ и более эффективное использование вычислительных ресурсов. Глубокое понимание архитектуры вычислительных систем позволяет осознанно выбирать подходящие уровни параллелизма и разрабатывать алгоритмы с учетом особенностей конкретной аппаратной платформы. Это особенно важно при создании высокоэффективного, масштабируемого и надежного программного обеспечения.

Настоящая книга посвящена основам параллельных вычислений на уровне задач и алгоритмов. В ней рассматриваются методы разработки параллельных программ на языке программирования C++ с использованием технологий MPI и OpenMP: основных средств построения вычислительных систем. Описы-

Черновик Погребняк М. А.

ваются принципы их применения, синтаксис, а также ключевые особенности. Приводятся примеры численных алгоритмов и рассматриваются методы их эффективного распараллеливания, а также представлены различные варианты реализации этих алгоритмов.

Черновик
Погребняк М. А.

Черновик
Погребняк М. А.

Основы параллельных вычислений

Принципы декомпозиции вычислительных задач

Прежде чем приступить к распараллеливанию вычислительной задачи, необходимо определить возможность ее эффективного параллельного выполнения. Для этого требуется провести анализ зависимостей в структуре данных задачи.

Под данными задачи в этом случае понимается совокупность различных объектов, над которыми выполняется вычислительный процесс в рамках алгоритма. Это могут быть, например, элементы матрицы, узлы и элементы сетки, а также массивы значений функций или состояний.

При распараллеливании алгоритма важно понимать, как организованы его данные и как они связаны между собой. Если каждый фрагмент данных может обрабатываться независимо от других, это означает, что в задаче присутствуют только локальные зависимости. Значит, данные такого алгоритма можно распределить между процессами или потоками. Если же между частями данных существуют глобальные зависимости, например, изменение одного элемента влияет на все остальные, распараллеливание становится невозможным без модификации алгоритма [11].

Таким образом, построение параллельной реализации вычислительной задачи требует учета взаимосвязи между структурой ее алгоритма и архитектурой целевой многопроцессор-

Черновик Погребняк М. А.

ной вычислительной системы. Такое согласование осуществляется на основе **уровней декомпозиции** — способов разбиения исходной задачи на независимые подзадачи для последующего распределения между вычислительными ресурсами [12]. Выбор конкретного уровня декомпозиции определяется вычислительной сложностью задачи и особенностями ее алгоритмической структуры.

Выделяют следующие основные уровни декомпозиции [13]:

Разбиение задачи на подзадачи — выделение в общей постановке отдельных компонентов, которые могут решаться независимо или с минимальной степенью взаимодействия.

Разбиение каждой отдельной подзадачи на множество процедур — построение набора одинаковых или схожих по структуре процедур, которые могут выполняться параллельно при различных входных данных.

Распараллеливание отдельных процедур — организация внутреннего параллелизма на уровне вычислений внутри одной процедуры.

Разбиение арифметических операций между процессорами — самый низкий уровень декомпозиции, предполагающий распределение отдельных элементарных операций (например, сложений, умножений) между процессами.

Простейший пример декомпозиции можно рассмотреть на задаче суммирования элементов числового ряда. Благодаря ассоциативности и коммутативности операции сложения, вычисления можно эффективно распределить между несколькими параллельными процессами, что ускорит общее время выполнения без нарушения логики алгоритма.

Пусть дан числовой ряд из N элементов и требуется вычислить его сумму с использованием p параллельных процессов или потоков:

$$S = \sum_{i=0}^{N-1} a_i.$$

В процессе декомпозиции исходная задача разбивается на p независимых подзадач таким образом, чтобы каждый процесс $k \in \{0, \ldots, p-1\}$ обрабатывал собственное подмножество

Черновик Потребяк М. А.

индексов $I_k \subseteq \{0, \ldots, N-1\}$, причем множества I_k попарно не пересекаются, а их объединение покрывает весь исходный диапазон:

$$\bigcup_{k=0}^{p-1} I_k = \{0, 1, \ldots, N-1\}, \quad I_i \cap I_j = \varnothing \text{ при } i \neq j.$$

Линейный диапазон индексов делится на равные (или почти равные) сегменты по числу доступных процессов. Границы сегмента для процесса k определяются по формулам:

$$s_k = \left\lfloor \frac{N \cdot k}{p} \right\rfloor, \qquad f_k = \left\lfloor \frac{N \cdot (k+1)}{p} \right\rfloor,$$

где процесс k обрабатывает элементы с номерами от s_k до f_k включительно.

Например, при $N = 10$, а $p = 3$ разбиение имеет вил:

для $k = 0$: $s_0 = \left\lfloor \frac{10 \cdot 0}{3} \right\rfloor = 0$, $f_0 = \left\lfloor \frac{10 \cdot 1}{3} \right\rfloor = 3$;
для $k = 1$: $s_1 = 3$, $f_1 = 6$;
для $k = 2$: $s_2 = 6$, $f_2 = 10$.

Тогда каждый процесс будет выполнять следующие действия:

процесс 0: $a_0 + a_1 + a_2$;
процесс 1: $a_3 + a_4 + a_5$;
процесс 2: $a_6 + a_7 + a_8 + a_9$.

После выполнения локального суммирования каждый процесс получает свое значение частичной суммы S_k, которые затем необходимо собрать, чтобы вычислить итоговую сумму:

$$S = \sum_{k=0}^{p-1} S_k.$$

Обычно для этого используется операция сбора данных, при которой все процессы отправляют свои результаты одному процессу, выполняющему финальную агрегацию. Эта операция создает незначительную нагрузку по сравнению с основными вычислениями при условии $p \ll N$.

Черновик Погребняк М. А.

Методы разработки параллельных программ

В настоящее время наибольшее распространение получили два подхода к построению параллельных программ: геометрический параллелизм и метод коллективного решения [11]. Каждый из этих подходов обладает своими особенностями, преимуществами и ограничениями, а их выбор определяется характером вычислительной задачи и структурой данных.

Геометрический параллелизм предполагает разбиение области задачи (например, пространства, сетки или массива) между процессами и может применяться, например, при решении задач численного интегрирования или уравнений в частных производных. В отличие от него, метод коллективного решения ориентирован на эффективную обработку большого количества независимых однотипных задач — например, в методе Монте-Карло или при перемножении матриц.

Геометрический параллелизм

Геометрический параллелизм реализует **статическую балансировку**: каждому процессору заранее назначается определенный участок, на котором он выполняет вычисления. Такой подход эффективен при равномерной структуре задачи и стабильной вычислительной сложности по всей области.

Основная идея метода заключается в разбиении области на непересекающиеся (или, при необходимости, частично перекрывающиеся) фрагменты. В каждом таком фрагменте вычисления на очередном временном слое выполняются независимо. Исключение составляет необходимость синхронизации на границах — для согласования значений в соседних областях. Такая синхронизация может осуществляться, например, посредством обмена граничными значениями между процессами в конце каждого временного шага.

Преимущество этого подхода заключается в его универсальности, понятной структуре и высокой эффективности при решении широкого класса задач. Он хорошо масштабируется на многопроцессорных архитектурах и особенно эффективно работает в задачах с регулярной и равномерной сеточной струк-

Черновик Потребняк М. А.

турой. При этом статическая балансировка нагрузки снижает накладные расходы на управление параллелизмом, что делает данный метод особенно подходящим для систем с большим числом процессов.

Метод коллективного решения

Метод коллективного решения использует **динамическую балансировку**: вычислительные задания распределяются между процессорами в процессе выполнения, по мере их освобождения. Этот подход предпочтителен при непредсказуемой или нерегулярной нагрузке, когда заранее определить объем работы для каждого процессора затруднительно.

Метод коллективного решения особенно эффективен при наличии большого числа мелких и независимых подзадач, которые не требуют обмена данными между собой. Благодаря этому можно исключить необходимость межпроцессной синхронизации, что существенно упрощает реализацию параллельного алгоритма и снижает накладные расходы.

Ключевое преимущество этого подхода является его высокая гибкость в распределении вычислительных ресурсов. Задачи динамически назначаются процессам по мере их освобождения, что способствует равномерной загрузке и обеспечивает хорошую масштабируемость, даже при существенной неоднородности подзадач. Тем не менее эффективность метода значительно снижается, если входные данные уже распределены между процессами на начальном этапе. В этом случае требуется предварительный сбор данных на управляющем процессе, а затем — возврат результатов исходным владельцам. Дополнительные шаги связаны с высокими затратами на передачу данных, увеличивают объем межпроцессного взаимодействия и могут стать узким местом всей вычислительной системы.

Оценка эффективности параллельных алгоритмов

Параллельные вычисления требуют специализированного программного обеспечения, способного обеспечить надежную

Черновик Погребняк М. А.

и производительную реализацию алгоритмов. Ключевым фактором эффективности многопроцессорных систем является качество параллельных программ, которые должны максимально использовать вычислительные ресурсы при минимальных накладных расходах.

Разработка эффективных параллельных программ значительно сложнее, чем создание последовательных. Это обусловлено не только архитектурной спецификой многопроцессорных систем, но и необходимостью учитывать такие аспекты, как синхронизация, обмен данными и балансировка нагрузки между вычислительными узлами. Создание программного обеспечения для параллельных систем остается одной из ключевых задач современной вычислительной науки [11].

Идеальная параллельная программа обладает равной длительностью исполнения всех параллельных ветвей, отсутствием простоев, вызванных ожиданием данных, и конфликтов при доступе к общим ресурсам. Важным показателем эффективности также является полное перекрытие всех этапов вычислений и эффективный обмен данными. Повысить производительность удается за счет укрупнения единиц распараллеливания, упрощения генерации параллельных процедур, предварительной подготовки исходных данных, а также распараллеливания самих алгоритмов генерации программных компонентов [14].

Процесс адаптации последовательного кода под архитектуру параллельных вычислительных систем включает несколько ключевых этапов: анализ общей структуры программы, выделение основного вычислительного ядра, локализация операций ввода-вывода, оценка потенциального параллелизма, выявление и минимизация точек перераспределения данных, преобразование циклов для использования параллельных конструкций, оптимизация доступа к кэш-памяти, а также переход от глобальной к локальной обработке данных [15].

С теоретической точки зрения, эффективность параллельного алгоритма определяется соотношением между последовательными и параллельными частями вычислений. Пусть доля последовательных операций в алгоритме составляет α, где $0 \leq \alpha \leq 1$, а время выполнения последовательного варианта алгоритма равно T_1. Тогда время выполнения параллельной версии алгоритма на p идентичных процессорах можно выра-

Чернович Потребенник А.

зить формулой:

$$T_p = \alpha T_1 + \frac{(1-\alpha)T_1}{p}.$$

На основе этого выражения вводится понятие **ускорения параллельного алгоритма** — отношения времени выполнения последовательной версии алгоритма к времени выполнения его параллельной версии:

$$S_p = T_1/T_p,$$

где T_1 — время выполнения оптимального однопоточного варианта алгоритма (или время выполнения на одном процессоре), а T_p — время выполнения на p процессорах.

График ускорения параллельного алгоритма наглядно иллюстрирует эффективность масштабирования задачи при увеличении числа задействованных процессоров. По оси абсцисс откладывается количество потоков или процессов p, а по оси ординат — ускорение алгоритма S_p. Такой график показывает, во сколько раз параллельная реализация алгоритма превосходит по скорости последовательную. Идеальным считается линейный рост ускорения: $S_p = p$, при котором увеличение числа потоков приводит к пропорциональному сокращению времени выполнения. В реальных условиях достижение такой зависимости возможно лишь в редких случаях.

При чрезмерном увеличении количества потоков может наблюдаться падение относительной эффективности из-за роста накладных расходов. По мере увеличения p график ускорения отклоняется от идеальной прямой $S_p = p$ и постепенно выходит на насыщение. Это связано с тем, что в любой задаче существует часть операций, которая не поддается распараллеливанию, а также с затратами на синхронизацию, координацию и обмен данными между потоками. Поэтому существует понятие эффективности параллельного алгоритма:

$$E_p = S_p/p.$$

Эффективность отражает степень рационального использования вычислительных ресурсов при заданном уровне параллелизма. Чем ближе E_p к единице, тем лучше масштабируется алгоритм, и тем выше качество его реализации.

Черновик Погребняк М. А.

С теоретической точки зрения, максимальное ускорение ограничено законом Амдала, согласно которому при наличии доли последовательных операций α предельное ускорение не может превышать:

$$S_p \leq 1/\alpha.$$

Таким образом, график ускорения параллельного алгоритма S_p позволяет не только оценить выигрыш от параллельного исполнения, но и выявить структурные ограничения алгоритма, препятствующие дальнейшему увеличению производительности.

Черновик
Погребняк М. А.

Архитектуры и принципы построения параллельных вычислительных систем

Параллельные вычислительные системы можно разделить на два больших класса по способу работы с памятью: системы с общей памятью (shared memory) и системы с распределенной памятью (distributed memory) [16].

Архитектура с общей памятью (рис. 1) устроена следующим образом: существует единое физическое устройство памяти (memory, MEM), объединенное в общее виртуальное адресное пространство, к которому имеют доступ несколько вычислителей (central processing unit, CPU). В современном контексте под вычислителем могут подразумеваться как отдельные процессоры, так и отдельные ядра многоядерного процессора (core).

Архитектура с общей памятью предполагает, что все вычислители (процессоры или ядра) имеют прямой и равноправный доступ к единому блоку памяти. В простейшем случае доступ к памяти осуществляется с одинаковыми задержками и одинаковыми затратами ресурсов. Если при этом все вычислительные элементы обладают одинаковыми характеристиками, тогда такая архитектура называется симметричной многопроцессорной системой (Symmetric Multiprocessing, SMP).

Примером архитектуры с общей памятью являются совре-

Черновик Погребняк М. А.

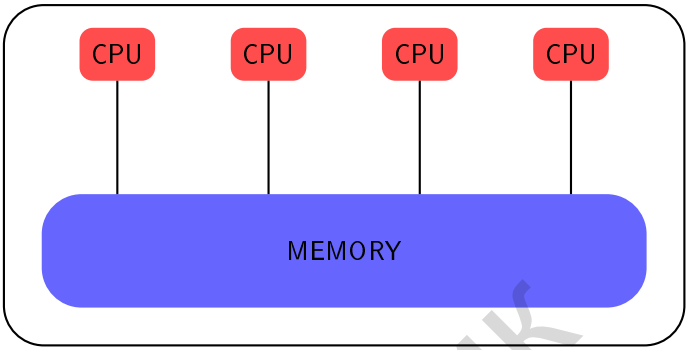

Рис. 1. Архитектура параллельной вычислительной системы с общей памятью.

менные многоядерные процессорные системы. Практически любое современное микропроцессорное устройство в той или иной степени реализует параллельную вычислительную архитектуру с общей памятью. Количество ядер в таких системах можно интерпретировать, на абстрактном уровне, как количество независимых процессоров.

Архитектура с распределенной памятью (рис. 2) отличается от архитектуры с общей памятью тем, что каждая вычислительная единица (процессор) имеет собственную физическую память. Иными словами, система состоит из множества пар процессор-память, и каждый процессор имеет прямой доступ только к своей памяти.

Такие системы также применяются для параллельных вычислений, при этом необходимо организовать эффективное взаимодействие между парами процессор–память. Для этого используется специальный коммуникационный слой, который чаще всего реализуется в виде сети или коммутатора, объединяющего все вычислительные узлы между собой (network). Этот слой обеспечивает передачу данных между различными частями системы и позволяет координировать выполнение распределенной задачи.

Первые компьютеры, предназначенные для параллельных вычислений, были построены на архитектуре с распределенной памятью. Каждый вычислительный узел представлял со-

Черновик Погребняк М. А.

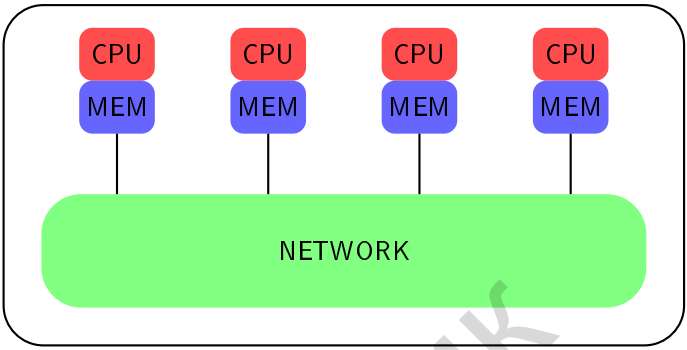

Рис. 2. Архитектура параллельной вычислительной системы с распределенной памятью.

бой отдельный автономный компьютер с одноядерным процессором, и все узлы соединялись между собой посредством сети.

В современных условиях отдельные компьютеры, как правило, являются многоядерными. Поэтому сейчас за каждым узлом системы с распределенной памятью фактически скрывается архитектура с общей памятью. Такие многоядерные узлы соединяются между собой через высокоскоростную сеть, что позволяет формировать масштабируемые вычислительные кластеры. Такой подход получил название гибридной архитектуры.

Гибридная архитектура — это подход, при котором вычислительные узлы с общей памятью объединяются в единую распределенную систему посредством сетевого взаимодействия. Такая архитектура сочетает в себе преимущества обоих типов организации: высокую скорость обмена данными внутри узлов за счет общей памяти и хорошую масштабируемость всей системы благодаря распределенной структуре.

Черновик Потребник М. А.

Черновик
Погребняк М. А.

Параллельное программирование с использованием MPI

Общие сведения о технологии MPI

В настоящее время одной из наиболее широко применяемых технологий программирования параллельных вычислений на компьютерах с распределенной памятью является технология MPI (Message Passing Interface, интерфейс передачи сообщений).

Основной механизм взаимодействия параллельных процессов в MPI — это передача сообщений между ними. Стандарт MPI задает набор интерфейсов, которым должны соответствовать как программные реализации на различных вычислительных платформах, так и пользовательские приложения, использующие этот интерфейс [17].

MPI — это не конкретная библиотека, а спецификация: она описывает набор функций, типов данных и других элементов, а также определяет их поведение. MPI предоставляет универсальный и эффективный стандарт для разработки параллельных приложений и поддерживается в языках C, C++ и Fortran.

Хотя интерфейс MPI допускает реализацию параллельных программ в стиле MIMD (Multiple Instruction, Multiple Data), при котором различные процессы могут выполнять разные программы, на практике такая модель используется редко из-за высокой сложности разработки и отладки. Значительно ча-

Черновик Попробняк М. А.

ще применяется модель SPMD (Single Program, Multiple Data), при которой все процессы запускают одну и ту же программу, но обрабатывают разные фрагменты данных.

Идея создания стандарта MPI возникла в начале 1990-х годов, когда стало очевидно, что отсутствие унифицированного подхода к параллельному программированию препятствует развитию высокопроизводительных вычислений. Разработкой и поддержкой стандарта занимается открытое сообщество MPI Forum [18]. На сегодняшний день MPI прошел несколько итераций развития, при этом каждая новая версия сохраняет обратную совместимость со всеми предыдущими:

1992 — Инициатива создания стандарта MPI;
1994 — MPI-1.0;
1996 — MPI-2.0;
2012 — MPI-3.0;
2021 — MPI-4.0.

На сегодняшний день стандарт MPI остается наиболее распространенным решением в области высокопроизводительных вычислений (High-Performance Computing, HPC) на системах с распределенной памятью. Другие технологии, разрабатываемые для параллельного программирования, как правило, не заменяют MPI, а дополняют его, используя в качестве внутренней подсистемы для организации обмена сообщениями между процессами. Более того, MPI может эффективно применяться и на системах с общей памятью, обеспечивая высокую производительность и масштабируемость.

MPI-программа представляет собой набор независимых процессов, которые взаимодействуют друг с другом посредством вызовов коммуникационных процедур. Ключевым понятием в модели программирования MPI является процесс.

Процесс — это исполняемая программа, обладающая собственным адресным пространством, ресурсами и контекстом выполнения. Все процессы в составе MPI-программы запускаются независимо друг от друга и не имеют прямого доступа к памяти других процессов.

Каждый MPI-процесс — это полноценный процесс операционной системы. Это принципиально отличает MPI от многопоточности: потоки существуют в рамках одного процесса

Черновик Погребняк М. А.

и делят общее адресное пространство, в то время как MPI ориентирован на взаимодействие между полностью изолированными процессами.

MPI реализует программную модель с явным управлением параллелизмом, в которой необходимо вручную определять структуру взаимодействия, обмена данными и синхронизации между процессами. В стандарте MPI-1 количество процессов задается заранее и остается неизменным на протяжении выполнения программы. Начиная с версии MPI-2, появилась возможность динамического создания и управления процессами во время исполнения.

Важной частью стандарта MPI являются механизмы организации взаимодействия между процессами. Все процессы в MPI организованы в группы, каждая из которых может быть связана с **коммуникатором** — специальным объектом, определяющим контекст взаимодействия.

Коммуникаторы необходимы для изоляции различных частей приложения и управления обменом сообщениями между процессами. Все функции MPI, связанные с взаимодействием между процессами, требуют указания коммуникатора. В большинстве базовых программ используется предопределенный коммуникатор `MPI_COMM_WORLD`, охватывающий все процессы текущего приложения.

Внутри коммуникатора каждому процессу присваивается уникальный идентификатор — rank (ранг), который представляет собой целое число от 0 до $N-1$, где N — число процессов. Ранг позволяет каждому процессу определить, какую часть работы он должен выполнить и с какими другими процессами взаимодействовать. Процесс с рангом 0 часто играет координирующую роль, управляя логикой обмена или сбором результатов.

Основной функционал стандарта MPI охватывает несколько ключевых направлений, обеспечивающих эффективную реализацию параллельных программ. Прежде всего, любая MPI-программа начинается с инициализации среды функцией `MPI_Init` и завершается вызовом `MPI_Finalize`. Межпроцессное взаимодействие реализуется через операции `MPI_Send` и `MPI_Recv`, позволяющие передавать данные между двумя конкретными процессами.

Черновик Потребник М. А.

Кроме того, стандарт предоставляет обширный набор коллективных операций, к которым относятся, например, широковещательная рассылка `MPI_Bcast`, сведение данных `MPI_Reduce` и синхронизация `MPI_Barrier`.

Для обмена данными MPI использует собственные типы, поскольку стандартные типы языков программирования могут по-разному интерпретироваться на различных аппаратных или программных платформах. В таблице 1 приведено соответствие предопределенных типов MPI стандартным типам язык программирования C++.

Тип MPI	**Тип языка C++**
MPI_CHAR	char
MPI_SHORT	short
MPI_INT	int
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short
MPI_UNSIGNED	unsigned int
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double

Таблица 1. Соответствие предопределенных типов MPI стандартным типам языка C++

Для удобства обмена данными в MPI можно использовать как предопределенные, так и пользовательские типы, что позволяет эффективно передавать не только базовые значения, но и сложные структуры.

Технология MPI реализует модель обмена сообщениями между процессами, которая не зависит от их физического расположения или операционной системы. Независимо от того, выполняются процессы на одном вычислительном узле или распределены по разным машинам в кластере, MPI обеспечивает единый и прозрачный механизм их взаимодействия через передачу сообщений.

Черновик Погребняк М. А.

Установка и настройка MPI на Windows с интеграцией в Visual Studio

Для разработки и запуска параллельных программ на технологии MPI в среде Windows необходимо установить Microsoft MPI и настроить его интеграцию с Microsoft Visual Studio.

MPI от Microsoft включает два установочных файла, которые можно скачать с официального сайта Microsoft [19]:

`msmpisetup.exe` — среда выполнения MPI (runtime);

`msmpisdk.msi` — комплект разработки программного обеспечения (SDK) для использования MPI.

Проверить корректность установки можно, открыв командную строку от имени администратора и выполнив команду:

```
set MSMPI
```

Если переменные среды `MSMPI_INC`, `MSMPI_BIN` и `MSMPI_LIB` отображаются корректно — установка прошла успешно.

Для работы проекта с использованием MPI в Visual Studio необходимо указать пути к заголовочным файлам в свойствах проекта. Для этого необходимо:

1. Перейти в
   `C/C++` → `General` → `Additional Include Directories`
2. Добавить путь
   `C:\Program Files (x86)\Microsoft SDKs\MPI\Include`

Далее необходимо указать компоновщику пути к библиотекам:

1. Перейти в
   `Linker` → `General` → `Additional Library Directories`
2. Добавить путь
   `C:\Program Files (x86)\Microsoft SDKs\MPI\Lib\x64`

А также подключить сами библиотеки:

1. Перейти в
   `Linker` → `Input` → `Additional Dependencies`
2. Добавить
   `msmpi.lib`, `msmpifec.lib`, `msmpifmc.lib`

Черновик Погребняк М. А.

Теперь можно собрать проект в Visual Studio:

`Build` → `Build Solution`

После сборки в папке `Debug` или `Release` будет создан исполняемый файл `<имя программы>.exe`.

После получения исполняемого файла его необходимо запустить на заданном количестве процессов. Для этого, как правило, используется команда запуска MPI-приложений `mpiexec` через командную строку:

```
mpiexec -n <N> <имя программы>.exe
```

где `N` — количество процессов. После запуска одна и та же программа будет выполняться всеми процессами параллельно.

Можно настроить запуск MPI-программ непосредственно из Visual Studio. Для этого необходимо:

1. Перейти в
   `Tools` → `External Tools...`
2. Нажать `Add`
3. Заполнить поля следующим образом:

   Title: `Run with MPI`
   Command:
   `C:\Program Files\Microsoft MPI\Bin\mpiexec.exe`
   Arguments: `-n <N> $(TargetName).exe`
   Initial Directory: `$(BinDir)`
4. Нажать `OK`

Теперь можно запускать программу на `N` процессах через:

`Tools` → `Run with MPI`

Базовые функции MPI

Рассмотрим простой пример MPI-программы, иллюстрирующий базовые принципы параллельного программирования.

Начнем с базового шаблона программы на языке C++:

Черновик Погребняк М. А.

```
#include<iostream>

/*
argc (argument count) - количество аргументов, переданных
при запуске программы
argv (argument vector) - массив строк (аргументов),
переданных через командную строку
*/
int main(int argc, char* argv[]) {
    std::cout << "Hello, world!\n";

    // Код возврата, сообщающий операционной системе, что
программа завершилась успешно.
    return 0;
}
```

Теперь модифицируем эту программу, используя технологию MPI. MPI-программа на C++ требует подключения заголовочного файла:

```
// Подключение библиотеки MPI
#include <mpi.h>
```

Обязательными элементами любой MPI-программы являются инициализация и корректное завершение MPI-окружения. Эти действия выполняются с помощью следующих функций:

```
// Инициализация MPI
MPI_Init(&argc, &argv);

// Завершение работы с MPI
MPI_Finalize();
```

Функция `MPI_Init` инициирует работу MPI и должна вызываться перед любыми другими функциями MPI. Она выполняет начальную настройку параллельной среды и подготавливает все процессы к обмену сообщениями. Вызов этой функции разрешен только один раз для каждого MPI-приложения. `MPI_Init` не создает новые процессы — они уже должны быть запущены заранее с помощью средства запуска MPI-программ.

При инициализации MPI создается группа всех процессов приложения, связанная с предопределенным коммуникатором `MPI_COMM_WORLD`. Эта область связи объединяет все процессы

Черновик Погребняк М. А.

и упорядочивает их, присваивая каждому уникальный номер (ранг).

Параметры командной строки `argc` и `argv` позволяют передавать в MPI-программу аргументы, которые необходимы для настройки поведения приложения при запуске.

Функция `MPI_Finalize` завершает работу MPI-программы, корректно закрывая все коммуникации и освобождая системные ресурсы, связанные с MPI. После ее вызова обращение к другим функциям MPI, включая `MPI_Init`, запрещено. До вызова этой функции каждый процесс должен завершить участие во всех коммуникационных операциях.

Для получения информации о процессах используются следующие функции:

```
int rank, size;

// Получить общее число процессов
MPI_Comm_size(comm, &size);

// Получить номер текущего процесса
MPI_Comm_rank(comm, &rank);
```

Функция `MPI_Comm_size` сохраняет в переменную `size` общее количество процессов в заданном коммуникаторе.

Функция `MPI_Comm_rank` записывает в переменную `rank` уникальный идентификатор (ранг) текущего процесса в пределах указанного коммуникатора.

На начальном этапе допустимыми значениями параметра `comm` являются только предопределенные коммуникаторы: `MPI_COMM_WORLD`, охватывающий все процессы текущего приложения, и `MPI_COMM_SELF`, включающий всего лишь один процесс — сам себя.

Модифицированная программа `Hello, world!` на C++ с использованием MPI:

```
#include <mpi.h>
#include <iostream>

int main(int argc, char* argv[]) {
    MPI_Init(&argc, &argv);

    int rank, size;
```

Черновик Погребняк М. А.

```
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    // Каждый процесс выводит свое сообщение
    std::cout
        << "Hello from process "
        << rank
        << " out of "
        << size
        << std::endl;

    MPI_Finalize();

    return 0;
}
```

При запуске программы на 4 процессах она выведет строки вида:

```
Hello from process 3 out of 4
Hello from process 2 out of 4
Hello from process 1 out of 4
Hello from process 0 out of 4
```

Порядок вывода может быть произвольным, поскольку процессы выполняются параллельно и не синхронизированы при выводе.

Работа с коммуникаторами в MPI

Во многих MPI-программах возникает необходимость ограничить область коммуникации определенным подмножеством процессов. Для этого создаются новые коммуникаторы, содержащие только нужные процессы. В MPI предусмотрены два тесно связанных механизма для этого: работа с группами процессов и создание соответствующих коммуникаторов.

Группа в MPI — это упорядоченное множество процессов, каждому из которых присвоен уникальный локальный идентификатор (ранг), начинающийся с 0. Для работы с группами используется специальный тип данных `MPI_Group` и набор соответствующих функций.

Существуют две предопределенные группы:

Черновик Потребняк М. А.

`MPI_GROUP_EMPTY` — пустая группа;

`MPI_GROUP_NULL` — специальное значение, обозначающее недействительную (неинициализированную) группу.

При инициализации `MPI_COMM_WORLD` соответствующая группа не создается автоматически, а должна быть получена явным образом с помощью специальной функции:

`MPI_Comm_group(MPI_Comm comm, MPI_Group *group)`.

Группы в MPI являются неизменяемыми: после создания их состав изменить нельзя, можно лишь сформировать новую группу на основе существующей.

Коммуникатор описывает область взаимодействия между процессами. Даже если два коммуникатора ссылаются на одну и ту же группу, они считаются различными, поскольку содержат независимые контексты обмена сообщениями.

Существуют два типа коммуникаторов:

`intracommunicator` — описывает взаимодействие внутри одной группы процессов;

`intercommunicator` — используется для организации связи между процессами двух разных групп.

После вызова `MPI_Init` создаются следующие предопределенные коммуникаторы:

`MPI_COMM_WORLD` — включает все процессы, участвующие в выполнении программы;

`MPI_COMM_SELF` — содержит только текущий процесс;

`MPI_COMM_NULL` — специальное значение, обозначающее неинициализированный или недопустимый коммуникатор.

Тип коммуникатора можно определить с помощью функции `MPI_Comm_test_inter`.

Функции для работы с коммуникаторами в MPI делятся на две основные категории:

Локальные (доступ) — не требуют обмена сообщениями между процессами:

Чернозик Погребняк М. А.

`MPI_Comm_size` — возвращает количество процессов в коммуникаторе;

`MPI_Comm_rank` — возвращает ранг (номер) текущего процесса в коммуникаторе;

`MPI_Comm_compare(comm1, comm2, &result)` — сравнивает два коммуникатора и определяет их эквивалентность.

Коллективные (создание) — требуют участия всех процессов коммуникатора:

`MPI_Comm_dup(comm, &newcomm)` — создает дубликат коммуникатора с теми же процессами;

`MPI_Comm_create(comm, group, &newcomm)` — создает новый коммуникатор для заданной группы процессов;

`MPI_Comm_split(comm, color, key, &newcomm)` — разбивает процессы на подгруппы в зависимости от значения `color`.

Удаление коммуникатора выполняется с помощью функции `MPI_Comm_free`.

Следующий пример на языке C++ демонстрирует разбиение 10 процессов на три подгруппы (`color`) с использованием `MPI_Comm_split`:

```
#include <mpi.h>
#include <iostream>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    // Глобальный коммуникатор
    MPI_Comm parent_comm = MPI_COMM_WORLD;
    int world_rank, world_size;
    MPI_Comm_rank(parent_comm, &world_rank);
    MPI_Comm_size(parent_comm, &world_size);

    // Назначение цвета группы: создается три группы (0,1,2)
    int color = world_rank % 3;

    // Создание подкоммуникатора по цвету
    MPI_Comm new_comm;
    MPI_Comm_split(parent_comm, color, world_rank,
&new_comm);
```

Черновик Потребняк М. А.

```
    // Получение информации в новом коммуникаторе
    int new_rank, new_size;
    MPI_Comm_rank(new_comm, &new_rank);
    MPI_Comm_size(new_comm, &new_size);

    // Вывод информации о текущем процессе
    std::cout
        << "Global rank "
        << world_rank
        << " -> New group "
        << color
        << ", rank in group "
        << new_rank
        << ", group size "
        << new_size
        << std::endl;

    // Освобождение ресурсов
    MPI_Comm_free(&new_comm);
    MPI_Finalize();
    return 0;
}
```

При запуске программы на 10 процессах она выведет строки вида:

```
Global rank 7 -> New group 1, rank in group 2, group size 3
Global rank 2 -> New group 2, rank in group 0, group size 3
Global rank 9 -> New group 0, rank in group 3, group size 4
Global rank 8 -> New group 2, rank in group 2, group size 3
Global rank 6 -> New group 0, rank in group 2, group size 4
Global rank 5 -> New group 2, rank in group 1, group size 3
Global rank 0 -> New group 0, rank in group 0, group size 4
Global rank 3 -> New group 0, rank in group 1, group size 4
Global rank 1 -> New group 1, rank in group 0, group size 3
Global rank 4 -> New group 1, rank in group 1, group size 3
```

После запуска программы каждый процесс получает информацию о своем глобальном ранге, номере подгруппы (`color`) и положении внутри этой подгруппы (локальный ранг и размер группы).

Черновик Погребняк М. А.

Взаимодействие типа точка-точка в MPI

Одной из базовых форм взаимодействия между процессами в MPI является обмен сообщениями типа точка-точка (point-to-point communication). В этом типе взаимодействия участвуют ровно два процесса: один выступает в роли отправителя, другой — в роли получателя.

В некоторых случаях отправитель и получатель могут совпадать, то есть процесс может отправлять сообщение самому себе. Формально такое взаимодействие все еще относится к обмену типа точка-точка, поскольку логически участвуют две стороны — отправитель и получатель, даже если это один и тот же процесс.

Главной особенностью взаимодействия типа точка-точка является явное управление передачей данных: каждый процесс должен точно указать, когда и что он отправляет или принимает.

Обмен сообщениями в MPI возможен только между процессами, принадлежащими одному коммуникатору, то есть находящимися в общем контексте взаимодействия. Если два процесса принадлежат разным коммуникаторам, они не могут напрямую обмениваться сообщениями. Это обеспечивает логическую изоляцию задач и позволяет организовывать независимые группы процессов в рамках одной программы.

Базовый интерфейс функций отправки и приема в MPI имеет следующий вид:

```
MPI_Send(void* buf, int count, MPI_Datatype datatype,
int dest, int tag, MPI_Comm comm);
MPI_Recv(void* buf, int count, MPI_Datatype datatype,
int source, int tag, MPI_Comm comm, MPI_Status* st);
```

Функция `MPI_Send` отправляет сообщение, состоящее из `count` элементов типа `datatype`, процессу с рангом `dest` в рамках коммуникатора `comm`. Параметр `tag` используется для маркировки сообщения, а указатель `buf` указывает на область памяти, содержащую отправляемые данные. Если передается одно значение, параметр `count` устанавливается равным 1.

Функция `MPI_Recv` осуществляет прием сообщения от процесса с рангом `source`, с заданной меткой `tag`, в рамках коммуникатора `comm`. Принятые данные записываются в буфер `buf`.

Черновик Погребняк М. А.

Параметр `st` содержит дополнительную информацию о принятом сообщении, такую как фактический размер и ранг отправителя. Для организации более гибкого приема допускается использовать специальные значения `MPI_ANY_SOURCE` и `MPI_ANY_TAG`, позволяющие принимать сообщение от любого источника и с любой меткой соответственно.

MPI поддерживает несколько режимов передачи сообщений, которые определяются типом используемой функции отправки. Все типы операций делятся на блокирующие и неблокирующие и отличаются по моменту завершения. Общим для всех режимов является гарантия корректности: завершение операции означает, что буфер можно безопасно использовать повторно, и любые изменения в данных больше не повлияют на противоположную сторону.

В таблице 2 приведены основные функции MPI для взаимодействия типа точка-точка, сгруппированные по режиму выполнения и характеру блокировки:

Режим передачи	**Блокирующие**	**Неблокирующие**
Стандартная	`MPI_Send`	`MPI_Isend`
Синхронная	`MPI_Ssend`	`MPI_Issend`
Буферизованная	`MPI_Bsend`	`MPI_Ibsend`
Согласованная	`MPI_Rsend`	`MPI_Irsend`
Прием сообщения	`MPI_Recv`	`MPI_Irecv`

Таблица 2. Коммуникационные функции типа точка-точка в MPI

Из таблицы 2 видно, что имена функций формируются по определенному правилу: к базовым операциям `MPI_Send` и `MPI_Recv` добавляются префиксы, отражающие особенности передачи данных.

Префикс `S` (Synchronous) — обозначает синхронный режим передачи.

Функция `MPI_Ssend` блокирует выполнение отправителя до тех пор, пока на стороне получателя не будет начат прием соответствующего сообщения. Иными словами, операция `Ssend` гарантированно не завершится, пока не будет вызвана `MPI_Recv`

Черновик Погребняк М. А.

с совпадающими параметрами `source` и `tag`. Такое поведение делает передачу синхронной: отправитель синхронизирован с получателем на момент начала приема. Функция `MPI_Ssend` не обязательно блокирует выполнение до полной передачи всех данных — она может завершиться сразу после того, как начнется прием.

Очевидный недостаток синхронной передачи — возможная блокировка. Например, если два процесса вызывают `MPI_Ssend` друг к другу, не вызывая при этом `MPI_Recv`, они оба будут бесконечно ожидать начала приема. В результате возникнет ситуация взаимной блокировки (deadlock). Поэтому использование синхронной передачи требует тщательного проектирования логики взаимодействия.

Префикс `B` (Buffered) — указывает на буферизованный режим передачи.

Процесс-отправитель вызывает `MPI_Bsend`, после чего MPI копирует передаваемые данные во внутренний буфер, который должен быть заранее создан вызовом `MPI_Buffer_attach`. Как только копирование завершено, `MPI_Bsend` немедленно завершается, независимо от того, готов ли процесс-получатель принять сообщение или нет.

Такой подход исключает блокировку и возможность взаимного зависания. Это достигается ценой дополнительных затрат: во-первых, требуется время на копирование данных в буфер; во-вторых, управлять буфером необходимо самостоятельно. Кроме того, размер буфера должен быть достаточным для размещения всех сообщений — в противном случае отправка завершится с ошибкой.

Префикс `R` (Ready) — указывает на согласованный (или подготовленный) режим передачи.

Операция передачи `MPI_Rsend` может быть выполнена только после того, как на принимающей стороне уже начат прием соответствующего сообщения. Если это условие не выполнено, поведение считается неопределенным и может привести к ошибке или аварийному завершению программы.

Главное преимущество передачи по готовности — минимальные накладные расходы. MPI может немедленно начать передачу данных без дополнительных проверок и буферизации,

Черновик Погребняк М. А.

если прием уже инициирован. Это достигается ценой жестких ограничений. Отправитель должен быть абсолютно уверен, что прием уже начат с соответствующими параметрами `source`, `tag` и `comm`.

Префикс `I` (Immediate) — обозначает неблокирующую форму вызова.

Функции с этим префиксом немедленно возвращают управление, даже если передача или прием еще не завершены. Для гарантированного завершения операции необходимо вызвать одну из функций контроля состояния, таких как `MPI_Wait` или `MPI_Test`. Эти функции позволяют явно определить момент, когда соответствующая операция передачи или приема действительно завершена.

Стандартная отсылка, реализуемая через функцию `MPI_Send`, представляет собой универсальный механизм передачи сообщений. Она может работать либо как синхронная, либо как буферизованная, в зависимости от реализации MPI, размера сообщения и состояния системы. Таким образом, стандартная отсылка пытается объединить достоинства обоих подходов: эффективность буферизованной передачи и надежность синхронной.

Такое поведение сопровождается возможной блокировкой и отсутствием строгих гарантий. Это делает стандартную отсылку непредсказуемой, особенно в сложных участках кода, где требуется высокая степень детерминированности. В таких случаях лучше использовать явно определенные типы пересылки.

В большинстве же случаев `MPI_Send` демонстрирует наилучшую производительность, так как MPI может автоматически выбирать оптимальный механизм передачи на основе текущих условий — например, размера сообщения или доступности ресурсов. Такой адаптивный подход позволяет добиться высокой эффективности, не перегружая программу ручным управлением стратегией пересылки.

В идеализированной модели, где данные передаются с постоянной скоростью и без каких-либо дополнительных накладных расходов, время передачи T пропорционально размеру со-

Черновик Погребняк М. А.

общения:

$$T = \beta N,$$

где N — размер сообщения, а β — время передачи одного байта. График (рис. 3) в этом случае — прямая, проходящая через начало координат. В реальных системах значительную роль играет фиксированная задержка на установку соединения — латентность (α), и общее время передачи моделируется как

$$T = \alpha + \beta N.$$

На графике (рис. 3) это прямая, смещенная вверх, отражающая влияние фиксированных затрат при малых объемах передачи.

Для оценки эффективности важно рассматривать пропускную способность:

$$R = \frac{N}{\alpha + \beta N}.$$

График пропускной способности (рис. 3) имеет форму вогнутой кривой: при малых N она близка к нулю (доминирует латентность), а при больших — стремится к максимуму $1/\beta$, что соответствует предельной скорости передачи. Это означает, что максимальная эффективность достигается только при передаче крупных сообщений.

Функция `MPI_Send` динамически адаптирует стратегию пересылки: при малых N она может использовать буферизацию, чтобы сгладить влияние латентности, а при больших — применять синхронную отсылку, чтобы избежать переполнения буфера.

Для всех типов взаимодействия в MPI используется единая функция приема — `MPI_Recv`. Она блокирует выполнение до полного получения сообщения и гарантирует, что данные записаны в указанный заранее буфер. Для асинхронного приема применяется `MPI_Irecv`, позволяющая продолжить выполнение программы до завершения приема.

При использовании блокирующих операций данные считаются доставленными, как только соответствующая функция завершилась: для `MPI_Send` — когда сообщение безопасно отправлено, для `MPI_Recv` — когда сообщение полностью получено. От одного отправителя к одному получателю сообщения

ЧЕРНОВИК
Погребняк М. А.

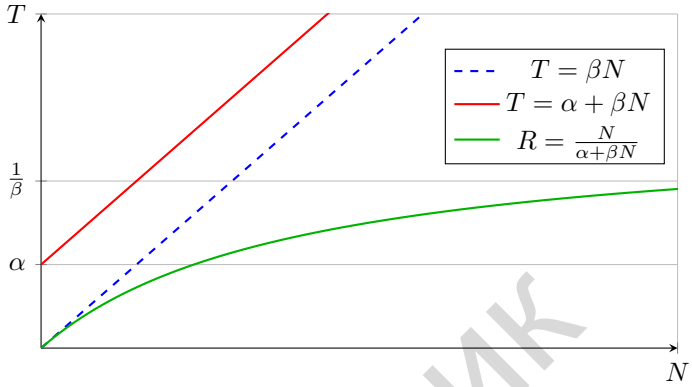

Рис. 3. Графики времени передачи и пропускной способности.

доставляются в порядке отправки, но между разными отправителями порядок поступления сообщений не гарантируется.

Дополнительные макросы и параметры, упрощающие реализацию:

- `MPI_ANY_SOURCE` — позволяет принять сообщение от любого процесса;
- `MPI_ANY_TAG` — позволяет принять сообщение с любым тегом;
- `MPI_STATUS_IGNORE` — позволяет проигнорировать информацию о статусе приема.

Примеры отправки данных

Передача одного числа:

```
int a = 56;
MPI_Send(&a, 1, MPI_INT, rank, tag, MPI_COMM_WORLD);
```

Передача статического массива:

```
int arr[5] = {1, 2, 3, 4, 5};

MPI_Send(arr, 5, MPI_INT, rank, tag, MPI_COMM_WORLD);
// или
MPI_Send(&arr[0], 5, MPI_INT, rank, tag, MPI_COMM_WORLD);
```

Черновик Погребняк М. А.

Передача динамического массива:

```
int* arr = new int[5]{10, 20, 30, 40, 50};
MPI_Send(arr, 5, MPI_INT, 1, 0, MPI_COMM_WORLD);
```

Передача строки:

```
char buf[64];
std::strcpy(buf, "Hello from process");
MPI_Send(buf, std::strlen(buf) + 1, MPI_CHAR, 1, 0,
MPI_COMM_WORLD);
```

Кольцевая пересылка:

```
#include <iostream>
#include <mpi.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    char buf[100];
    std::snprintf(buf, sizeof(buf), "Hello from %d", rank);

   // Синхронный режим передачи вызовет deadlock
   // MPI_Ssend(buf, 100, MPI_CHAR, (rank + 1) % size, 0,
MPI_COMM_WORLD);

    MPI_Send(buf, 100, MPI_CHAR, (rank + 1) % size, 0,
MPI_COMM_WORLD);
    MPI_Recv(buf, 100, MPI_CHAR, (rank - 1 + size) % size,
0, MPI_COMM_WORLD, MPI_STATUSES_IGNORE);

    std::cout
        << "Process "
        << rank
        << " received: "
        << buf
        << std::endl;

    MPI_Finalize();
    return 0;
}
```

Черновик Потребняк М. А.

При запуске программы на 10 процессах она выведет строки вида:

```
Process 7 received: Hello from 6
Process 6 received: Hello from 5
Process 8 received: Hello from 7
Process 3 received: Hello from 2
Process 4 received: Hello from 3
Process 5 received: Hello from 4
Process 1 received: Hello from 0
Process 2 received: Hello from 1
Process 9 received: Hello from 8
Process 0 received: Hello from 9
```

Коллективные операции в MPI

Помимо базовых форм взаимодействия типа точка-точка, MPI предоставляет большой набор коллективных операций, упрощающих реализацию часто встречающихся коммуникационных шаблонов. Все процессы, входящие в заданный коммуникатор, обязаны участвовать в коллективной операции — в противном случае программа завершится с ошибкой или зависнет.

Среди коллективных операций в MPI можно выделить: рассылку данных от одного процесса всем остальным (`MPI_Bcast`); сбор данных со всех процессов в один процесс (`MPI_Gather`, `MPI_Gatherv`); разбиение данных и отправку их разным процессам (`MPI_Scatter`, `MPI_Scatterv`); попарный обмен фрагментами между всеми участниками (`MPI_Alltoall`, `MPI_Alltoallv`), а также синхронизацию выполнения с помощью барьера (`MPI_Barrier`) [17].

Для глобальных вычислений над распределенными данными используются специальные редукционные операции, позволяющие, например, вычислить сумму, максимум или минимум значений, собранных со всех процессов.

Одной из ключевых коллективных операций в MPI является функция `MPI_Reduce`, предназначенная для выполнения редукции данных — объединения значений, полученных от всех процессов, с помощью заданной операции:

```
MPI_Reduce(void* sendbuf, void* recvbuf, int count,
MPI_Datatype dt, MPI_Op op, int root, MPI_Comm comm);
```

Черновик Погребняк М. А.

Функция `MPI_Reduce` выполняет операцию редукции над данными, распределенными между всеми процессами указанного коммуникатора, и передает результат одному из них. Параметр `sendbuf` указывает на буфер, содержащий локальные данные, над которыми будет производиться редукция. Параметр `recvbuf` используется только в root-процессе и задает буфер, в который будет записан результат.

Параметр `count` определяет количество элементов в массиве, участвующих в операции, а `dt` — тип передаваемых данных. Параметр `op` указывает тип выполняемой редукции (таблица 3). Аргумент `root` задает ранг процесса, в который будет помещен результат. А параметр `comm` определяет коммуникатор, в рамках которого происходит операция, то есть группу участвующих процессов.

Название	**Операция**	**Разрешенные типы**
`MPI_MAX`	Максимум	integer, real
`MPI_MIN`	Минимум	integer, real
`MPI_SUM`	Сумма	integer, real
`MPI_PROD`	Произведение	integer, real
`MPI_LAND`	Логическое AND	integer, boolean
`MPI_LOR`	Логическое OR	integer, boolean
`MPI_LXOR`	Логическое XOR	integer, boolean

Таблица 3. Операции редукции

Одним из типичных применений операции редукции является вычисление суммы чисел, распределенных между несколькими процессами. Предположим, требуется найти сумму всех целых чисел от 1 до некоторого значения N. Каждому процессу выделяется свой участок диапазона, на котором он вычисляет частичную сумму. Затем все частичные суммы объединяются в процессе с рангом 0 с помощью функции `MPI_Reduce`.

```
#include <mpi.h>
#include <iostream>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
```

Черновик учебника М. А.

```
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    const int64_t N = static_cast<int64_t>(INT_MAX);
    int64_t local_sum = 0;

    // Делим диапазон между процессами.
    int64_t chunk_size = N / size;
    int64_t start = rank * chunk_size + 1;
    int64_t end = (rank == size - 1)
        ? N
        : start + chunk_size - 1;

    for (int64_t i = start; i <= end; ++i) {
        local_sum += i;
    }

    int64_t global_sum = 0;

    // Суммируем все локальные суммы.
    MPI_Reduce(&local_sum, &global_sum, 1, MPI_LONG_LONG,
MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        std::cout
        << "The sum of numbers from 1 to "
        << N
        << " is "
        << global_sum
        << std::endl;
    }

    MPI_Finalize();
    return 0;
}
```

Каждый процесс независимо вычисляет сумму чисел на своем участке, что позволяет распределить вычислительную нагрузку. Затем частичные суммы объединяются на процессе с рангом 0, где и получается итоговый результат.

Важно отметить, что при делении диапазона на участки последний процесс обрабатывает хвост диапазона, если N

Черновик Погребняк М. А.

не делится нацело на количество процессов. Это гарантирует корректность и полноту расчетов.

При запуске программы на 10 процессах она выведет строку вида:

```
The sum of numbers from 1 to 2147483647 is
2305843008139952128
```

Черновик
Погребняк М. А.

Черновик
Погребняк М. А.

Параллельное программирование с использованием OpenMP

Общие сведения о технологии OpenMP

В настоящее время наиболее широко используемой технологией параллельного программирования для систем с общей памятью является OpenMP (Open Multi-Processing, открытая мультипроцессорная обработка).

Технология OpenMP базируется на концепции совместного использования общего адресного пространства несколькими потоками исполнения (нитями). Программа, использующая OpenMP, запускается как единый процесс, который может порождать несколько потоков. Каждый поток выполняет одну и ту же программу, но обрабатывает разные участки данных, что соответствует модели SPMD (Single Program Multiple Data) [20].

OpenMP не является библиотекой в традиционном смысле, а представляет собой спецификацию, определяющую стандарт параллельного программирования. Она реализуется в виде директив компилятора, встроенных функций и переменных окружения. OpenMP поддерживается компиляторами для языков C, C++ и Fortran, и реализуется на всех основных операцион-

Черновик Погребняк М. А.

ных системах.

Первоначально OpenMP разрабатывался как закрытый стандарт с целью автоматического распараллеливания последовательных программ. Впоследствии он был открыт и передан в ведение открытой международной организации OpenMP Architecture Review Board [21]. Подобно MPI, спецификация OpenMP прошла через множество версий и продолжает активно развиваться при участии университетов, исследовательских центров и ведущих технологических компаний:

1997 — OpenMP 1.0 для Fortran;

1998 — OpenMP 1.0 для C/C++;

2000 — OpenMP 2.0 для Fortran;

2002 — OpenMP 2.0 для C/C++;

2008 — OpenMP 3.0;

2013 — OpenMP 4.0;

2018 — OpenMP 5.0;

2023 — OpenMP 6.0.

OpenMP реализует модель параллельного программирования **fork and join** (разветвление и слияние), при которой основной поток исполнения (master thread) при входе в параллельную область выполняет операцию fork — создает группу подчиненных потоков (worker threads). Эти рабочие потоки параллельно исполняют обозначенный участок кода. По завершении параллельной области происходит операция join, в ходе которой все рабочие потоки завершают свою работу, и выполнение возвращается к главному потоку. Таким образом, исполнение программы чередует последовательные и параллельные участки (рис. 4).

Последовательные секции — участки кода, исполняемые только главным потоком. К ним относятся, например, инициализация данных до входа в параллельную область, вывод результатов и другие операции, не подлежащие распараллеливанию.

Параллельные секции — участки кода, исполняемые всеми потоками одновременно, внутри которых каждый поток может обрабатывать собственную часть данных.

Черновик Погребняк М. А.

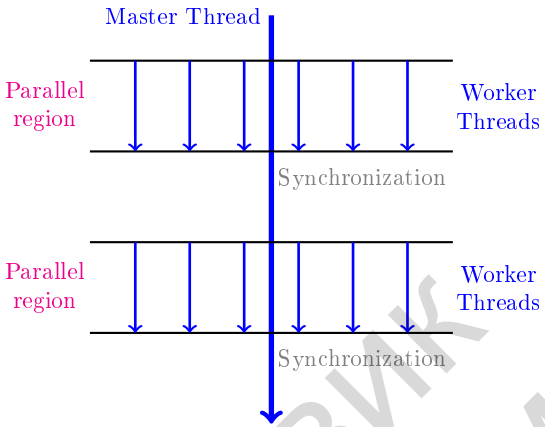

Рис. 4. Fork and Join модель

Особенностью OpenMP является то, что потоки всегда создаются группой, а не индивидуально. Это отличает OpenMP от классического многопоточного программирования, например, с использованием `std::thread` в C++ или `POSIX Threads` в C [22], где потоки создаются и управляются вручную по одному. В OpenMP управление созданием, распределением и синхронизацией потоков делегируется компилятору и среде выполнения.

Создание и завершение потоков в OpenMP — это сравнительно быстрые операции благодаря использованию облегченных механизмов управления, встроенных в среду выполнения. Тем не менее частое выполнение операций fork и join, особенно внутри циклов или при вложенных параллельных участках, может приводить к заметным задержкам. Основным источником накладных расходов является барьерная синхронизация: операция join требует, чтобы основной поток ожидал завершения всех порожденных потоков, прежде чем продолжить выполнение программы.

Для снижения накладных расходов в современных реализациях OpenMP используется механизм повторного использования потоков (thread pooling). Вместо того чтобы создавать и уничтожать потоки при каждом входе и выходе из парал-

Черновик Погребняк М. А.

лельной секции, потоки создаются один раз при первом входе в параллельную область и затем переводятся в режим ожидания. При следующем входе в параллельную область эти потоки повторно активируются без повторного создания, что позволяет существенно ускорить выполнение программы.

Одним из ключевых преимуществ OpenMP является простота параллелизации кода, которая достигается путем добавления специальных директив компилятора, что позволяет минимизировать усилия, затрачиваемые на переход от последовательной модели к параллельной. Кроме того, благодаря автоматическому управлению созданием, синхронизацией и завершением потоков, OpenMP ускоряет разработку и отладку по сравнению с ручным управлением потоками.

Несмотря на преимущества, OpenMP имеет ряд ограничений, обусловленных архитектурой и моделью fork and join. OpenMP предназначен только для систем с общей памятью и не поддерживает межпроцессное взаимодействие, как, например, MPI. Все потоки работают в одном адресном пространстве, что может привести к состояниям гонки при совместном доступе к данным.

При использовании OpenMP важно учитывать, что операции ввода-вывода, выполняемые из разных потоков, могут происходить в произвольном порядке. Это связано с тем, что потоки выполняются параллельно и конкурируют за доступ к стандартным потокам ввода и вывода. В результате сообщения от разных потоков могут перемешиваться и отображаться на экране в непредсказуемой последовательности.

Настройка OpenMP на Windows с интеграцией в Visual Studio

Для разработки и запуска параллельных программ с использованием технологии OpenMP в среде Microsoft Visual Studio не требуется установка дополнительных компонентов. Встроенный компилятор Visual Studio обеспечивает полную поддержку стандарта OpenMP.

Для работы с технологией OpenMP в Visual Studio достаточно активировать ее поддержку в настройках проекта:

Черновик Погребняк М. А.

1. Перейти в:
   `C/C++` → `Language`
2. Включите опцию `OpenMP Support` (установить значение `Yes`)

Для проверки поддержки OpenMP и определения доступной версии можно воспользоваться следующим кодом:

```
#include <iostream>

int main(int argc, char** argv) {

    #ifdef _OPENMP
    std::cout
        << "OpenMP is supported! Version: "
        << _OPENMP
        << std::endl;
    #else
    std::cout
        << "OpenMP is NOT supported!"
        << std::endl;
    #endif

    return 0;
}
```

Значение макроса `_OPENMP` имеет формат `yyyymm`, где `yyyy` обозначает год, а `mm` — месяц утверждения поддерживаемого стандарта. Например, значение `202011` соответствует версии OpenMP 5.1, принятой в ноябре 2020 года.

Библиотечные процедуры OpenMP

Для использования функций OpenMP необходимо подключить заголовочный файл, который содержит объявления всех основных процедур и макросов OpenMP:

```
// Подключение функций OpenMP
#include <omp.h>
```

Ниже приведены основные функции времени выполнения, предоставляемые OpenMP, а также их назначение:

Черновик Погребняк М. А.

Функция `int omp_get_num_threads()` — возвращает количество потоков, участвующих в выполнении текущей параллельной области. Если вызов производится вне параллельной области, результатом будет 1.

Функция `int omp_get_thread_num()` — возвращает уникальный номер текущего потока в пределах параллельной области. Нумерация начинается с нуля. Если функция вызывается в последовательной области, возвращается 0.

Функция `int omp_get_num_procs()` — возвращает количество процессоров (или логических ядер), доступных для выполнения потоков. Это значение можно использовать для настройки оптимального числа потоков.

Функция `void omp_set_num_threads(int n)` — задает желаемое количество потоков, которые будут использоваться для выполнения параллельных областей. Эта установка применяется только к тем параллельным областям, которые начинаются после вызова данной функции. Если `omp_set_num_threads()` не вызывалась, количество потоков может быть задано переменной окружения `OMP_NUM_THREADS` или определяться системой автоматически.

Функция `double omp_get_wtime()` — возвращает текущее значение времени в секундах, отсчитываемое от некоторого фиксированного момента (обычно — времени запуска программы). Может использоваться для измерения времени выполнения участков кода. Например, разность между значениями, возвращенными до и после выполнения фрагмента, показывает продолжительность выполнения этого фрагмента.

Функция `double omp_get_wtick()` — возвращает длительность системного такта в секундах, то есть минимальное различимое изменение значений, возвращаемых `omp_get_wtime()`. Это значение может служить для оценки погрешности при измерении времени.

Директивы и модель данных в OpenMP

Основной механизм технологии OpenMP — это директивы компилятора, позволяющие описывать параллельные участки программы без явного управления потоками. **Директива** — это

Черновик Погребняк М. А.

специальная инструкция компилятору, указывающая, как обрабатывать следующий оператор или блок кода.

Все директивы OpenMP начинаются с конструкции `#pragma omp`, за которой следует имя конкретной директивы (`directive`) и, при необходимости, одна или несколько опций `clause`, определяющих поведение переменных и потоков. Директива применяется к одному оператору или блоку кода, заключенному в фигурные скобки:

```
#pragma omp directive [clause ...]
```

OpenMP охватывает различные аспекты параллельного программирования и использует директивы, которые можно условно разделить на три функциональные группы.

Первая группа включает директивы, предназначенные для создания и управления параллельными областями. К ним относятся `parallel`, `sections`, `task`, `single`, `master` и другие конструкции, позволяющие запускать независимые участки кода в нескольких потоках.

Вторая группа предназначена для распределения работы между потоками. Сюда относятся директивы `for` и `sections`, а также механизмы управления стратегией распределения итераций, такие как `schedule`, `collapse`, `reduction` и другие.

Третья группа охватывает средства синхронизации. Она включает директивы `critical`, `atomic`, `barrier`, `ordered`, `taskwait`, `nowait` и другие, обеспечивающие согласованность выполнения, защиту общих ресурсов и корректное взаимодействие между потоками.

Ключевая особенность OpenMP заключается в том, что управление параллелизмом и областью видимости переменных осуществляется непосредственно на уровне директив. **Модель данных** в OpenMP предполагает четкое разграничение памяти на общую (shared) и локальную (private).

Общие переменные (shared) доступны всем потокам, задействованным в параллельной области, и могут использоваться как для чтения, так и для записи. Локальные переменные (private) создаются независимо для каждого потока и существуют только в пределах времени жизни соответствующей параллельной секции. По умолчанию переменные, объявленные вне параллельного блока, считаются общими, а переменные,

Черновик Погребняк М. А.

объявленные внутри — локальными. При необходимости область видимости переменных может быть явно указана с помощью соответствующих квалификаторов. Например, директива `#pragma omp parallel private(x) shared(y)` указывает, что переменная `x` будет локальной для каждого потока, а переменная `y` — общей.

Для более точного контроля используются следующие спецификаторы переменных в директивах OpenMP:

`private(var)` — для каждого потока создается независимая копия переменной `var`, но начальное значение не копируется из оригинала;

`shared(var)` — переменная `var` становится доступна всем потокам, работает как ссылка на общую память;

`firstprivate(var)` — аналогична `private`, но копия инициализируется значением переменной до входа в параллельную область;

`lastprivate(var)` — аналогична `private`, но значение из последней итерации (или последнего исполнившего потока) сохраняется в оригинальную переменную после завершения параллельной области;

`reduction(op:var)` — создает локальные копии переменной `var` для каждого потока и объединяет их по завершении с использованием оператора `op` (например, `+`, `*`).

Грамотное применение этих спецификаторов позволяет избежать состояний гонки, снизить накладные расходы на синхронизацию и повысить эффективность выполнения параллельных программ.

Директива `parallel`

Наиболее базовой директивой в OpenMP является `parallel`. Она указывает компилятору, что следующий блок кода должен быть выполнен в параллельном режиме:

```
#pragma omp parallel
{
    // Параллельный код
}
```


Черновик Погребняк М. А.

При входе в параллельную секцию выполняется операция fork: основной (главный) поток создает группу рабочих потоков. Каждый из них начинает выполнение кода внутри параллельного блока. По завершении этой секции выполняется операция join: потоки синхронизируются, завершаются, и управление передается обратно основному потоку. По умолчанию OpenMP создает количество потоков, равное числу доступных логических ядер.

Директива `parallel` поддерживает следующие опции:

`if(condition)` — выполнение секции в параллельном или последовательном режиме в зависимости от переданного логического условия;

`num_threads(n)` — явное указание числа потоков;

`shared(varlist)` — объявление переменных, доступных всем потокам;

`private(varlist)` — объявление переменных, для которых каждый поток создает свою локальную не инициализированную копию;

`firstprivate(varlist)` — копии переменных инициализируются значениями из главного потока;

`reduction(operator:varlist)` — указание переменных, объединяемых по завершении секции при помощи заданной операции;

`default(shared|none)` — установка поведения по умолчанию для переменных, не указанных явно.

Директива `parallel` применяется к одному блоку или оператору, непосредственно следующему за ней. Важно учитывать, что открывающая фигурная скобка не должна находиться на той же строке, что и `#pragma` — это может привести к ошибке на этапе препроцессинга.

Пример параллельной секции:

```
#include <iostream>
#include <omp.h>

int main(int argc, char** argv) {
```

Черновик Погребняк М. А.

```
    #pragma omp parallel
    {
        std::cout << "Hello, world!\n";
    }

    return 0;
}
```

На многопроцессорной системе вывод повторится столько раз, сколько доступно логических ядер. Для явного указания количества потоков можно использовать опцию `num_threads`:

```
#pragma omp parallel num_threads(4)
{
    // Код выполняется 4 потоками
}
```

Для установки опции `num_threads` можно использовать выражения:

```
int n = 4;
#pragma omp parallel num_threads(n + 2)
```

Помимо опций директив, количество потоков в OpenMP можно задать с помощью функции `omp_set_num_threads(int n)`, которая устанавливает число потоков глобально для всех последующих параллельных участков (если явно не указано иное):

```
#include <iostream>
#include <omp.h>

int main(int argc, char** argv) {

    omp_set_num_threads(4);

    #pragma omp parallel
    {
        std::cout << "Hello, world!\n";
    }

    return 0;
}
```

Альтернативным способом задания числа потоков является установка переменной окружения `OMP_NUM_THREADS`, например, через терминал перед запуском программы:

Черновик Погребняк М. А.

```
set OMP_NUM_THREADS=2 && <имя программы>.exe
```

Порядок приоритета источников, определяющих количество потоков, следующий:

Значение по умолчанию — соответствует количеству доступных логических ядер.

Переменная окружения `OMP_NUM_THREADS`.

Вызов `omp_set_num_threads(n)` в коде.

Опция `num_threads(...)` в директиве `#pragma omp parallel`.

Ниже приведен пример, иллюстрирующий, какой из методов имеет приоритет, если заданы несколько источников одновременно.

```
#include <iostream>
#include <omp.h>

int main(int argc, char** argv) {

    #pragma omp parallel
    {
        std::cout
            << "Parallel region 1\n";
    }

    omp_set_num_threads(2);

    #pragma omp parallel num_threads(3)
    {
        std::cout
            << "Parallel region 2\n";
    }

    #pragma omp parallel
    {
        std::cout
            << "Parallel region 3\n";
    }

    return 0;
}
```

Черновик Потребняк М.А.

Первая секция использует значение заданное умолчанию, вторая — явно заданное число потоков с помощью опции `num_threads`, а третья — число потоков, установленное с помощью функции `omp_set_num_threads`. При запуске программы она выведет строки следующего вида:

```
Parallel region 1
Parallel region 1
Parallel region 1
Parallel region 1
Parallel region 1
Parallel region 1
Parallel region 1
Parallel region 1
Parallel region 2
Parallel region 2
Parallel region 2
Parallel region 3
Parallel region 3
```

Для получения информации о потоках используются функции `omp_get_thread_num()`, которая возвращает номер текущего потока от 0 до $N-1$, и `omp_get_num_threads()`, которая возвращает общее число потоков в текущей параллельной области.

```
#include <iostream>
#include <omp.h>

int main(int argc, char** argv) {

    #pragma omp parallel
    {
        int rank = omp_get_thread_num();
        int size = omp_get_num_threads();

        printf(
            "Thread %d out of %d\n",
            rank,
            size);
    }

    return 0;
}
```

Черновик Погребняк М. А.

При запуске программы она выведет строки следующего вида:

```
Thread 0 out of 8
Thread 6 out of 8
Thread 2 out of 8
Thread 4 out of 8
Thread 5 out of 8
Thread 3 out of 8
Thread 1 out of 8
Thread 7 out of 8
```

Видимости переменных

Область видимости переменных в OpenMP зависит от места их объявления и используемых опций. При доступе нескольких потоков к общей переменной без синхронизации возникает состояние гонки, приводящее к непредсказуемым результатам.

```
#include <iostream>
#include <omp.h>

int main(int argc, char** argv) {

    int a_shared = 5;

    #pragma omp parallel
    {
        int rank = omp_get_thread_num();
        printf(
            "Thread %d sees a_shared = %d\n",
            rank,
            a_shared);

        a_shared = 5 * rank;
    }

    return 0;
}
```

В этом примере переменная `a_shared` — общая, и каждый поток может перезаписать ее значение. Поскольку записи выполняются асинхронно, итоговое состояние переменной неопределено:

Черновик Погребняк М. А.

```
Thread 0 sees a_shared = 5
Thread 1 sees a_shared = 5
Thread 3 sees a_shared = 0
Thread 4 sees a_shared = 5
Thread 7 sees a_shared = 15
Thread 2 sees a_shared = 5
Thread 5 sees a_shared = 20
Thread 6 sees a_shared = 35
```

Таким образом, директива `parallel` служит основой распараллеливания в OpenMP и предоставляет гибкий способ запуска многопоточного исполнения с возможностью управления числом потоков, поведением переменных и стратегиями распределения данных.

Независимые секции

Если различные фрагменты кода не зависят друг от друга и могут выполняться одновременно, в OpenMP для их параллельного исполнения используется конструкция `sections`. Каждую независимую часть кода оформляют с помощью директивы `section`. Такая структура позволяет явно задать фиксированное количество задач, которые будут распределены между потоками и выполнены параллельно.

```
#include <iostream>
#include <omp.h>

void func1() { std::cout << "A\n"; }
void func2() { std::cout << "B\n"; }

int main() {
    #pragma omp parallel sections
    {
        #pragma omp section
        func1();

        #pragma omp section
        func2();
    }

    return 0;
}
```

Черновик Погребняк М. А.

В приведенном примере функции `func1()` и `func2()` выполняются параллельно — каждая в своей секции. OpenMP создает группу потоков, каждый из которых может быть назначен на выполнение одной из секций. Количество секций должно быть задано явно. Если число доступных потоков превышает количество секций, оставшиеся потоки останутся неиспользованными в этой параллельной области.

Директивы `critical`, `atomic`, `single`, `master`

В OpenMP предусмотрены директивы, предназначенные для управления выполнением отдельных участков кода внутри параллельной секции: `critical`, `atomic`, `single`, `master`.

Директива `critical`

Директива `critical` применяется для организации критической секции — фрагмента кода, который в каждый момент времени может выполняться только одним потоком. Остальные потоки при этом приостанавливаются и ждут своей очереди на вход в секцию. Такая синхронизация необходима для предотвращения состояния гонки (race condition), возникающего при одновременном доступе потоков к общим переменным.

```
#include <iostream>
#include <omp.h>

int main(int argc, char** argv) {

    int count = 0;

    #pragma omp parallel
    {
        #pragma omp critical
        {
            ++count;
        }
    }
```

Черновик Погреонак М. А.

```
    std::cout << count;

    return 0;
}
```

Если программа выполняется с четырьмя потоками, то итоговое значение счетчика всегда будет равно 4: каждый поток последовательно входит в критическую секцию и увеличивает значение на единицу.

Чрезмерное использование критических секций снижает эффективность параллельного выполнения, так как приводит к последовательной обработке этих участков кода. Поэтому их следует применять только в тех случаях, когда действительно требуется синхронизация доступа к общим ресурсам.

В задачах, сводящихся к простой агрегации значений (например, суммирование, поиск минимума или максимума), вместо критической секции предпочтительнее использовать опцию `reduction` в директиве `parallel`. Она автоматически создает локальные копии переменных для каждого потока и безопасно объединяет результаты по завершении параллельной области.

```
#include <iostream>
#include <omp.h>

int main(int argc, char** argv) {

    int count = 0;

    #pragma omp parallel reduction(+:count)
    {
        count++;
    }

    std::cout << count;

    return 0;
}
```

Для каждой переменной, указанной в списке после директивы `reduction`, создаются отдельные локальные копии в каждом потоке. По завершении параллельной области все локальные значения объединяются с использованием указанного оператора редукции.

Черновик Погребняк М. А.

Директива `reduction` поддерживает следующие операторы: `+`, `*`, `-`, `&`, `|`, `^`, `&&`, `||`.

Следует учитывать, что порядок выполнения агрегирующих операций не определен. Поэтому при использовании неассоциативных или непереместительных операций (например, вычитания или логических операций) результат может оказаться недетерминированным и зависеть от конкретного распределения итераций между потоками.

Директива `atomic`

Директива `atomic` применяется к простым операциям с общими переменными, таким как `x += y`. Она гарантирует атомарность записи в левую часть выражения, обеспечивая корректную синхронизацию при параллельном доступе.

Директива применима только к определенным типам выражений: составным арифметическим операциям, операциям инкрементам/декрементам и операциям присваивания. При этом атомарность распространяется только на операцию записи, для вычислений в правой части атомарность не гарантирована.

```
#pragma omp atomic
x += y;
```

Директива `atomic` является более эффективной альтернативой `critical` для простых операций, так как ее накладные расходы на синхронизацию значительно ниже, особенно при большом числе потоков.

Директива `single`

Для того чтобы определенный участок кода был выполнен ровно одним потоком (не обязательно главным), используется директива `single`. По умолчанию остальные потоки ожидают завершения этого участка. Если такое ожидание не требуется, его можно отключить с помощью опции `nowait`.

```
#pragma omp single
{
    // Это фрагмент выполнит один из потоков
}
```

Черновик Погребняк М. А.

Директиву `single` удобно применять для операций, которые должны выполняться только один раз, независимо от числа потоков. Это может быть, например, вывод информации на экран, инициализация общих данных, запись в файл и другие действия, которые не требуют многократного выполнения.

Директива `master`

Директива `master` указывает, что следующий участок кода должен быть выполнен исключительно основным (нулевым) потоком. В отличие от директивы `single`, здесь не происходит неявной синхронизации — остальные потоки просто пропускают этот участок и продолжают выполнение программы без ожидания.

```
#pragma omp master
{
    // Это фрагмент выполнит только главный поток
}
```

Директива `master` применяется, когда необходимо, чтобы определенное действие выполнялось исключительно главным потоком, например, при взаимодействии с внешней системой, управлении вводом-выводом, или реализации логики, зависящей от потока управления верхнего уровня.

Директива `for`

Одним из ключевых способов эффективного использования многопоточности в вычислительно интенсивных приложениях является параллельное выполнение циклов. В OpenMP для этого применяется директива `#pragma omp for` или ее комбинированная форма `#pragma omp parallel for`, которая автоматически распределяет итерации цикла между потоками.

Каждый поток получает часть итераций и выполняет их независимо от других. Распределение итераций происходит автоматически — либо на уровне компилятора, либо в среде выполнения OpenMP. По умолчанию после завершения всех итераций цикла происходит барьерная синхронизация: все потоки

Черновик Погребняк М. А.

ожидают завершения друг друга перед продолжением выполнения программы.

```
const int N = 10;
int A[N];

#pragma omp parallel for
for (int i = 0; i < N; ++i) {
    A[i] = i * i;
}
```

Поведение параллельного цикла можно детально настроить с помощью различных опций:

`collapse(n)` — объединяет `n` вложенных циклов в одно итерационное пространство;

`nowait` — отключает барьерную синхронизацию;

`ordered` — обеспечивает сохранение порядка выполнения;

`schedule(type[, chunk])` — задает стратегию распределения итераций между потоками. Параметр `type` определяет тип стратегии, а `chunk` указывает количество итераций в одном блоке.

Параметр `schedule` директивы `for` определяет, как итерации цикла распределяются между потоками, и тем самым позволяет гибко управлять балансом нагрузки и производительностью параллельного участка. Доступны следующие стратегии:

`static[, chunk]` — статическое распределение. Все итерации разбиваются на блоки фиксированной длины `chunk` и назначаются потокам в порядке их создания. Если `chunk` не задан, итерации делятся равномерно по числу потоков. Такой подход эффективен при равномерной нагрузке и минимизирует накладные расходы на планирование.

`dynamic[, chunk]` — динамическое распределение. Потоки получают блоки итераций по `chunk` штук по мере освобождения. Это полезно при неравномерной нагрузке между итерациями, поскольку позволяет лучше использовать ресурсы, но за счет динамического планирования увеличиваются накладные расходы.

Черновик Погребняк М. А.

`guided[, chunk]` — направленное распределение. Первоначально потоку выдаются крупные блоки, но по мере выполнения блоки становятся все меньше. Такой подход обеспечивает компромисс между равномерностью загрузки и накладными расходами. Параметр `chunk` задает минимальный размер блока (по умолчанию — 1).

`auto` — стратегия выбирается компилятором или OpenMP. Это дает компилятору свободу принимать оптимальные решения на основе архитектуры и контекста.

`runtime` — стратегия и размер блока определяются во время выполнения программы с помощью переменной окружения `OMP_SCHEDULE`.

Например, стратегия `schedule(static, 20)` означает, что итерации цикла будут разбиты на блоки по 20 штук, и эти блоки будут распределяться между потоками в фиксированном (статическом) порядке. Такая схема хорошо подходит для равномерно распределенной нагрузки, так как снижает накладные расходы на планирование.

Стратегия `schedule(dynamic, 20)` указывает, что блоки по 20 итераций будут динамически назначаться потокам по мере освобождения. Это эффективно при неоднородной нагрузке, так как позволяет лучше балансировать работу между потоками.

```
#include <iostream>
#include <omp.h>

int main() {
    const int N = INT_MAX / 10;

    double* a = new double[N];
    double* b = new double[N];
    double* c = new double[N];

    #pragma omp for schedule(auto)
    for (int i = 0; i < N; ++i) {
        a[i] = i;
    }

    double static_time = omp_get_wtime();
```

Черновик Погребняк М. А.

```
    #pragma omp parallel
    {
        #pragma omp for schedule(static, 20)
        for (int i = 0; i < N; ++i) {
            b[i] = a[i] * a[i];
        }
    }

    std::cout << "Static time: "
        << (omp_get_wtime() - static_time)
        << std::endl;

    double dynamic_time = omp_get_wtime();
    #pragma omp parallel
    {
        #pragma omp for schedule(dynamic, 20)
        for (int i = 0; i < N; ++i) {
            c[i] = a[i] * a[i];
        }
    }

    std::cout
        << "Dynamic time: "
        << (omp_get_wtime() - dynamic_time)
        << std::endl;

    return 0;
}
```

При запуске программы она выведет строки следующего вида:

```
Static time: 0.543517
Dynamic time: 0.560587
```

Черновик Потребняк М. А.

Черновик
Погребняк М. А.

Методы параллельной реализации численных алгоритмов

Перемножение матриц

Рассмотрим возможности параллельной реализации численных методов линейной алгебры на примере умножения двух матриц. Задачи линейной алгебры являются частыми для вычислительной математики и составляют основу многих алгоритмов — от решения систем линейных алгебраических уравнений до вычисления собственных значений и векторов, а также используются в задачах оптимизации, машинного обучения и моделирования различных физических процессов.

Рассмотрим две матрицы $A \in \mathbb{R}^{n \times m}$ — матрица размером $n \times m$ и $B \in \mathbb{R}^{m \times p}$ — матрица размером $m \times p$. Необходимо вычислить их произведение $C = A \cdot B$, где результирующая матрица C имеет размер $n \times p$, а каждый ек элемент c_{ij} вычисляется по следующей формуле:

$$c_{ij} = \sum_{k=1}^{m} a_{ik} b_{kj}, \quad 1 \leq i \leq n, 1 \leq j \leq p.$$

В последовательной реализации используется три вложенных цикла, каждый из которых проходит по одной размерности. Итерации этих циклов охватывают все возможные комбинации индексов i, j и k, соответствующие строкам и столбцам входных матриц:

Черновик Потребник М. А.

Последовательный алгоритм умножения матриц

```
for i = 0 to n − 1 do
    for j = 0 to p − 1 do
        for k = 0 to m − 1 do
            C[i][j] = C[i][j] + A[i][k] · B[k][j]
        end for
    end for
end for
```

Каждая строка результирующей матрицы C вычисляется независимо от других строк, что позволяет эффективно распараллелить задачу. Одним из способов декомпозиции в этом случае является разделение строк между потоками. Пусть используется p потоков. Тогда поток с номером $k \in 0, 1, \ldots, p-1$ будет обрабатывать определенный диапазон строк результирующей матрицы C.

Параллельный алгоритм умножения матриц в этом случае будет выглядеть следующим образом:

Параллельный алгоритм умножения матриц

```
Параллельно для каждого потока k = 0 . . . p − 1.
for i = s_k to f_k − 1 do
    for j = 0 to p − 1 do
        for l = 0 to m − 1 do
            C[i][j] = C[i][j] + A[i][l] · B[l][j]
        end for
    end for
end for
```

Здесь s_k и f_k — индексы начала и конца диапазона строк, обрабатываемых потоком с номером k. Такая декомпозиция обеспечивает равномерное распределение строк между потоками и предотвращает наложение или пропуск участков данных.

Численное интегрирование

Рассмотрим возможности параллельной реализации методов редукции на примере задачи численного интегрирования. За-

Черновик Погребняк М. А.

дачи редукции предполагают получение скалярного значения из набора векторных данных, например: суммы, максимума или произведения. Такие операции часто встречаются в задачах агрегации данных — при вычислении среднего значения, дисперсии, подсчете элементов, а также, в частности, при приближенном интегрировании.

Рассмотрим функцию $f(x)$, для которой требуется найти определенный интеграл на отрезке $[a, b]$:

$$I = \int_a^b f(x)\,dx.$$

Для приближенного вычисления интеграла можно использовать метод прямоугольников, при котором отрезок $[a, b]$ разбивается на N равных интервалов длины $h = (b - a)/N$. Тогда приближенное значение интеграла вычисляется по формуле:

$$I \approx h \sum_{i=0}^{N-1} f\left(x_i + \frac{h}{2}\right), \quad x_i = a + ih.$$

Таким образом вычисление интеграла сводится к суммированию значений функции на равномерной сетке. В последовательной реализации значения $f(x_i)$ вычисляются по шагам и накапливаются в переменной суммы:

Последовательный алгоритм численного интегрирования

```
I = 0
for i = 0 to N − 1 do
    x_i = a + ih + h/2
    I = I + f(x_i)
end for
I = I · h
```

Последовательный подход содержит цикл, который можно эффективно распараллелить. Благодаря слабой зависимости по данным каждый процесс может независимо вычислять сумму на своем участке интегрирования. Для этого применяется декомпозиция по геометрическому признаку — область интегрирования разбивается на равные подотрезки, которые распределяются между процессами. Каждый процесс выполняет

Черновик Погребняк М. А.

суммирование значений функции на своем участке, после чего частичные результаты объединяются в одно итоговое значение:

Параллельный алгоритм численного интегрирования

$I_k = 0$
for $k = 0$ **to** $N - 1$ **step** p **do**
  $x = a + i \cdot h + \frac{h}{2}$
  $I_k = I_k + f(x)$
end for
Выполнить редукцию: $I = h \cdot \sum_{k=0}^{p-1} I_k$

Уравнение теплопроводности

Рассмотрим возможности параллельной реализации явных сеточных методов на примере одномерного уравнения теплопроводности. Явные схемы широко применяются при численном решении гиперболических и параболических уравнений, особенно в задачах, не требующих решения системы линейных уравнений на каждом временном шаге.

Рассмотрим одномерное уравнение теплопроводности при постоянных шагах по пространству и времени:

$$\begin{aligned}
&\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}, \\
&u(x, 0) = \varphi(x), \\
&u(0, t) = u_L(t), \\
&u(L, t) = u_R(t), \\
&0 \le x \le L, \quad 0 \le t \le T,
\end{aligned}$$

где $u(x, t)$ — температура в точке x в момент времени t, $\alpha > 0$ описывает коэффициент температуропроводности, $\varphi(x)$ — начальное распределение температуры, а $u_L(t)$ и $u_R(t)$ — заданные граничные условия на концах отрезка.

Для численного решения необходимо провести дискретизацию, использовав трехточечную конечно-разностную схему первого порядка, то есть необходим переход от непрерывной постановки к сеточной, при котором дифференциальные операторы заменяются их конечно-разностными аналогами.

Черновик Погребняк М. А.

Область по пространственной координате разбивается на N равных интервалов с шагом $h = L/N$:

$$x_n = nh, \quad n = 0, 1, \ldots, N.$$

Временной интервал $[0, T]$ разбивается на M шагов с шагом $\tau = T/M$:

$$t^m = m\tau, \quad m = 0, 1, \ldots, M.$$

Приближенное значение функции $u(x_n, t^m)$ обозначим как u_n^m. В таком случае явная схема имеет следующий вид:

$$u_n^{m+1} = u_n^m + \frac{\alpha\tau}{h^2}(u_{n-1}^m - 2u_n^m + u_{n+1}^m).$$

Начальные и граничные условия в дискретной форме задаются следующим образом:

$$u_b^0 = \varphi(xbi), \quad u_0^m = u_L(t^m), \quad u_N^m = u_R(t^m).$$

В явной схеме на каждом временном шаге необходимо обновить значения температуры в пространственных узлах. Для хранения значений используются два массива: один содержит данные на текущем временном слое, другой — предназначен для записи значений на следующем временном слое. После завершения вычислений на шаге массивы меняются местами.

Ниже приведен краткий последовательный алгоритм численного решения уравнения теплопроводности:

Последовательный алгоритм уравнения теплопроводности

for $m = 0$ **to** M **do**
  for $n = 0$ **to** N **do**
    $u_n^{m+1} = u_n^m + \frac{\alpha\tau}{h^2}(u_{n-1}^m - 2u_n^m + u_{n+1}^m)$
  end for
  $u^m = u^{m+1}$
end for
Сохранить u^m

Структура алгоритма состоит из двух вложенных циклов: внешний цикл отвечает за продвижение по времени, а внутренний — за обход пространственных узлов. Поскольку каждая итерация внешнего цикла зависит от результатов предыдущей, его невозможно распараллелить напрямую. В то же

Черновик Попченко М. А.

время внутренний цикл поддается параллелизации, так как на каждом временном шаге значения в узлах вычисляются только на основе соседних значений, то есть зависимости являются локальными.

Благодаря такой локальной структуре зависимостей (каждое значение u_n^{m+1} зависит только от соседних значений на предыдущем временном слое) вычисления можно эффективно распараллеливать по пространственным координатам, применяя декомпозицию внутреннего цикла по геометрическому признаку.

Для корректного расчета значений на границе каждому процессу необходим доступ к данным, которые хранятся или вычисляются в соседнем процессе. Для решения этой проблемы применяется декомпозиция с перекрытием областей: каждый процесс, помимо основной зоны ответственности, хранит по одному дополнительному узлу с каждой стороны — так называемые приграничные узлы.

Таким образом, каждый процесс выполняет локальные вычисления в своей области, предварительно обменявшись необходимыми граничными значениями с соседями. Перед началом каждого временного шага выполняется синхронизация: процессы отправляют свои граничные данные соседним процессам и получают соответствующие значения взамен.

Ниже приведен краткий параллельный алгоритм численного решения уравнения теплопроводности:

Параллельный алгоритм уравнения теплопроводности

---

for $m = 0$ **to** M **do**
  Отправить приграничные узлы соседним процессам
  Принять приграничные узлы от соседних процессов
  for $n = 0$ $s_k \dots f_k$ **do**
    $u_n^{m+1} = u_n^m + \frac{\alpha\tau}{h^2}(u_{n-1}^m - 2u_n^m + u_{n+1}^m)$
  end for
  $u^m = u^{m+1}$
end for
Собрать данные со всех процессов
Сохранить u^m

Черновик Потребяк М. А.

Вопросы для самопроверки

Общие вопросы о параллельных вычислениях

1. Что такое параллельные вычисления и в каких областях они применяются?
2. Какие существуют основные уровни параллелизма? Приведите примеры для каждого.
3. Какие принципы декомпозиции вычислительных задач используются в параллельном программировании?
4. В чем заключается разница между геометрическим параллелизмом и методом коллективного решения?
5. Как оценивается эффективность параллельного алгоритма? Как построить его график?
6. Что такое ускорение параллельного алгоритма и как оно вычисляется?
7. В чем суть закона Амдала и каково его значение для параллельных вычислений?
8. Как организовано взаимодействие между процессами в системах с распределенной памятью?
9. Как организовано взаимодействие между потоками в системах с общей памятью?
10. В чем заключается разница между потоками и процессами?

Черновик Погребняк М. А.

Технология MPI

1. В чем заключается суть технологии MPI и для каких вычислительных систем она предназначена?
2. Какова структура и общая организация MPI?
3. Какие типы данных поддерживает MPI?
4. Что такое коммуникаторы и группы в MPI, в чем их сходство и различие?
5. Какие функции используются для работы с коммуникаторами?
6. Какие существуют режимы передачи сообщений в MPI?
7. В чем особенности функции `MPI_Send`?
8. В чем особенности функции `MPI_Recv`?
9. Какие блокирующие операции типа точка-точка предоставляет MPI?
10. Какие неблокирующие операции типа точка-точка предоставляет MPI?
11. Что такое буферизованная передача сообщений?
12. Как организовать кольцевую пересылку данных между процессами?
13. Какие функции используются для коллективных операций?
14. В чем особенности функции `MPI_Reduce`?
15. Какие типы операций редукции поддерживает MPI?

Технология OpenMP

1. В чем заключается суть технологии OpenMP и для каких вычислительных систем она предназначена?
2. Какова структура и общая организация OpenMP?
3. Как работает модель многопоточности fork and join?
4. Какие типы переменных используются в OpenMP, и в чем заключаются их различия и сходства?
5. Чем отличается главный поток от порожденных потоков?
6. Как задать количество потоков в OpenMP-программе?

Черновик Погребняк М. А.

7. Какие директивы используются для создания параллельных областей?
8. Как использовать директиву `parallel` в OpenMP?
9. Для чего используется опция `reduction`?
10. Для чего используются директивы `sections` в OpenMP?
11. Какие директивы применяются для синхронизации потоков?
12. Для чего используются директивы `critical` и `atomic`?
13. В чем различие между `single`, `master` и `critical`?
14. Как распараллелить цикл с помощью директивы `#pragma omp for`?
15. Какие стратегии планирования задаются с помощью опции `schedule`?

Черновик
Погребняк М. А.

Черновик
Погребняк М. А.

Список литературы

[1] Д Макарский и А Никоноров. *История компьютерной эры*. 2016.

[2] В Гергель. *Теория и практика параллельных вычислений*. 2016.

[3] А Волосова. *Параллельные методы и алгоритмы*. 2020.

[4] G Kamiadakis и R Kirby. *Parallel Scientific Computing in C++ and MPI*. 2003.

[5] Tom Deakin и Timothy G Mattson. “Programming your GPU with OpenMP”. В: *Context* 249 (2023), с. 251.

[6] М Восс, Р Асенхо и Д Рейндерс. *Параллельное программирование на C++ с помощью библиотеки TBB*. 2020.

[7] Р Гримм. *Параллельное программирование на современном языке C++*. 2022.

[8] Ш Танвар. *Параллельное программирование на C# и .NET Core*. 2021.

[9] Г Брайан. *Параллельное программирование в JAVA на практике*. 2020.

[10] J Palach. *Parallel programming with Python*. 2014.

[11] P Pacheco. *An introduction to parallel programming*. 2021.

[12] *Введение в проблематику разработки параллельных программ*. `https://pvs-studio.ru/ru/blog/posts/a0016/`. Accessed: 2025-07-01.

[13] М Якобовский. *Введение в параллельные методы*. 2013.

[14] В Костенко. “К вопросу об оценке оптимальной степени параллелизма”. В: *Программирование* 4 (1995), с. 24—28.

Черновик Потребняк М. А.

[15] А Антонов и В Воеводин. “Эффективная адаптация последовательных программ для современных векторно конвеерных и массивно параллельных супер-ЭВМ”. В: *Программирование* 4 (1996), с. 37—51.

[16] В Соснин и др. *Введение в параллельные вычисления.* 2023.

[17] Е Акимова. *Параллельные вычисления.* 2023.

[18] *MPI corporation.* `https://www.mpi-corporation.com`. Accessed: 2025-07-01.

[19] *Microsoft MPI.* `https://learn.microsoft.com/en-us/message-passing-interface/microsoft-mpi`. Accessed: 2025-07-01.

[20] Е Акимова и В Мисилов. *Параллельные вычисления на суперкомпьютерах с помощью технологии OpenMP.* 2023.

[21] *OpenMP Architecture Review Board.* `https://www.openmp.org`. Accessed: 2025-07-01.

[22] R Johnson. *Boost. Thread in Practice: Definitive Reference for Developers and Engineers.* 2025.

Черновик
Погребняк М. А.

Для заметок

Черновик
Погребняк М. А.

Для заметок

Черновик
Погребняк М. А.

Учебное издание

Погребняк Максим Анатольевич

Многопоточные вычисления на основе технологий MPI и OpenMP

Учебное пособие

Редактор, корректор Л. Н. Селиванова
Компьютерный набор и верстка М. А. Погребняк

Подписано в печать 01.09.2025. Формат 60×84/16.
Усл. печ. л. 4,0. Уч.изд. л. 3,0.
Тираж 16 экз.

Отпечатано ООО «Филигрань»
150049, Ярославль, ул. Свободы, 91.
pechataet@bk.ru

Черновик Погребняк М. А.

Черновик
Погребняк М. А.